

Useful
Code
Snippets

ULTIMATE LANGCHAIN GUIDE

Langchain Cheatsheet

Updated with latest LangChain 1.0 version



Chains



Agents



RAG



Memory



Tools



LCEL



Naved Khan

Senior Gen-AI Engineer

1/26

+ Follow

Basic Setup - Part 1

What is Basic Setup? This is the foundation of LangChain - directly talking to an LLM (like ChatGPT) with a prompt. ChatOpenAI is the wrapper that connects LangChain to OpenAI chat models.

- ✓ temperature controls randomness (low = factual, high = creative)
- ✓ .invoke() accepts messages (list) or a single string
- ✓ Best for quick experiments, demos, or one-off prompts

```
from langchain_openai import ChatOpenAI

chat = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0.7
)
```

PYTHON



Naved Khan
Senior Gen-AI Engineer

2/26

+ Follow

Basic Setup - Part 2

Continue: Using the chat model with different input formats.

PYTHON

```
# Using string (converted to HumanMessage automatically)
response = chat.invoke("What is LangChain?")
print(response.content)

# Using messages format (recommended)
from langchain_core.messages import HumanMessage
response = chat.invoke([HumanMessage(content="What is
LangChain?")])
print(response.content)
```



Naved Khan
Senior Gen-AI Engineer

3/26

+ Follow

Prompt Template

What is Prompt Template? Used to reuse prompts with dynamic values. Helps avoid hardcoding text into your code. Supports placeholders like {topic}, {concept}, etc.

- ✓ Makes prompts clean, readable, and maintainable
- ✓ Ideal when the prompt structure stays same but inputs change

```
from langchain_core.prompts import PromptTemplate PYTHON  
  
prompt = PromptTemplate.from_template(  
    "Write a poem about {topic}"  
)  
print(prompt.format(topic="Python"))
```



Naved Khan
Senior Gen-AI Engineer

4/26

+ Follow

ChatPromptTemplate

What is ChatPromptTemplate? Allows system + human + AI role-based prompts. Mimics how ChatGPT conversations work internally. Useful for instruction-following and persona control.

- ✓ Keeps prompts structured instead of long strings
- ✓ Commonly used in production-grade chains

```
from langchain_core.prompts import ChatPromptTemplate  PYTHON

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an expert developer."),
    ("human", "Explain {topic} with a simple example.")
])
formatted = prompt.format_messages(topic="APIs")
print(formatted)
```



Naved Khan
Senior Gen-AI Engineer

5/26

+ Follow

Structured Output - Part 1

What is Structured Output? Forces the model to return output in a specific format (JSON, schema). Very useful when output needs to be parsed by code. Reduces hallucinated or inconsistent responses.

- ✓ Uses Pydantic models for validation

```
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage
from pydantic import BaseModel, Field

class Person(BaseModel):
    name: str
    age: int
    summary: str = Field(description="Brief summary")
    category: str
```

PYTHON



Naved Khan
Senior Gen-AI Engineer

6/26

+ Follow

Structured Output - Part 2

Continue: Create structured model and use it to get validated output.

PYTHON

```
model = ChatOpenAI(model="gpt-4o-mini")
structured_model = model.with_structured_output(Person)

# Using messages format (recommended)
result = structured_model.invoke([
    HumanMessage(content="Name: Alice, Age: 28. Provide
summary and category.")
])
print(result)
```



Naved Khan
Senior Gen-AI Engineer

7/26

+ Follow

LCEL Chains - Part 1

What is LCEL? LCEL = LangChain Expression Language. Uses | to connect components like Unix pipes. The modern way to build chains in LangChain. Each step does one job (prompt → model → parser).

✓ Standard approach for all LangChain applications

```
PYTHON
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import
StrOutputParser
from langchain_openai import ChatOpenAI

# Build chain using LCEL pipe operator
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)
```



Naved Khan
Senior Gen-AI Engineer

8/26

+ Follow

LCEL Chains - Part 2

Continue: Build and invoke the chain.

PYTHON

```
chain = (  
    ChatPromptTemplate.from_messages([  
        ("system", "You are a helpful assistant."),  
        ("human", "Tell me a joke about {topic}")  
    ])  
    | llm  
    | StrOutputParser()  
)  
  
# Invoke the chain  
result = chain.invoke({"topic": "Python"})  
print(result)
```



Naved Khan
Senior Gen-AI Engineer

9/26

+ Follow

Agents and Tools - Part 1

What are Agents? Agents decide which tool to use and when. Tools are normal Python functions exposed to the LLM. Agent reasons step-by-step (ReAct pattern). Useful when the task requires decision-making, not just text.

✓ Common use cases: calculators, search, APIs, file operations

PYTHON

```
from langchain.tools import tool
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI

# Define a tool using @tool decorator
@tool
def calculator(expression: str) → str:
    """Do basic math. Input should be a valid
    mathematical expression."""
    try:
        result = eval(expression)
        return str(result)
    except Exception as e:
        return f"Error: {str(e)}"
```

10/26



Naved Khan
Senior Gen-AI Engineer

+ Follow

Agents and Tools - Part 2

Continue: Create agent with tools and invoke it.

PYTHON

```
# Create agent with tools (using model string or
ChatModel instance)
agent = create_agent(
    model="gpt-4o-mini", # Can use model string
    directly
    tools=[calculator],
    system_prompt="You are a helpful assistant that can
do math."
)

# Invoke agent with messages
result = agent.invoke({
    "messages": [{"role": "user", "content": "What is 5
* 12?"}]
})
print(result["messages"][-1].content)
```



Naved Khan
Senior Gen-AI Engineer

11/26

+ Follow

Memory - Part 1

What is Memory? Stores conversation history across multiple interactions. Allows the model to remember previous messages within a thread. Makes chats feel stateful and human-like.

- ✓ Uses checkpoints to persist state between conversations

```
from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver

# Create agent with memory (checkpointer)
checkpointer = InMemorySaver()
agent = create_agent(
    model="gpt-4o-mini",
    tools=[],
    checkpointer=checkpointer
)
```

PYTHON



Naved Khan
Senior Gen-AI Engineer

12/26

+ Follow

Memory - Part 2

Continue: Configure and use memory across multiple interactions.

PYTHON

```
# Configuration for thread persistence
config = {"configurable": {"thread_id": "1"}}

# First message
result1 = agent.invoke(
    {"messages": [{"role": "user", "content": "My name
is Alice"}]},
    config
)

# Second message - agent remembers the name from
previous interaction
result2 = agent.invoke(
    {"messages": [{"role": "user", "content": "What's my
name?"}]},
    config
)
print(result2["messages"][-1].content)
```

13/26



Naved Khan
Senior Gen-AI Engineer

+ Follow

Document Loading

What are Document Loaders? Converts files into LangChain Document objects. Supports TXT, PDF, HTML, CSV, etc. First step for RAG (Retrieval-Augmented Generation). Usually followed by chunking and embeddings.

- ✓ Separates data ingestion from reasoning

PYTHON

```
# TEXT FILES
from langchain_community.document_loaders import
TextLoader

docs = TextLoader("notes.txt").load()

# PDF FILES
from langchain_community.document_loaders import
PyPDFLoader

docs = PyPDFLoader("book.pdf").load()
```



Naved Khan
Senior Gen-AI Engineer

14/26

+ Follow

Embeddings + Vector Store - Part 1

What are Embeddings and Vector Stores? Embeddings convert text into numerical vectors. Vector stores enable semantic search. FAISS is a fast in-memory vector database.

✓ Core foundation of RAG systems

PYTHON

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_community.document_loaders import
TextLoader
from langchain_text_splitters import
RecursiveCharacterTextSplitter

# Load and split documents
loader = TextLoader("notes.txt")
documents = loader.load()
```



Naved Khan
Senior Gen-AI Engineer

15/26

+ Follow

Embeddings + Vector Store - Part 2

Continue: Split documents, create embeddings and search.

PYTHON

```
text_splitter =  
RecursiveCharacterTextSplitter(chunk_size=1000,  
chunk_overlap=0)  
chunks = text_splitter.split_documents(documents)  
  
# Create embeddings and vector store  
embeddings = OpenAIEmbeddings(model="text-embedding-3-  
small")  
vector_store = FAISS.from_documents(chunks, embeddings)  
  
# Search for similar documents  
results = vector_store.similarity_search("Explain  
LangChain", k=3)  
print(results)
```



Naved Khan
Senior Gen-AI Engineer

16/26

+ Follow

RAG Pipeline - Part 1

What is RAG? Combines retriever + LLM automatically.

Retrieves relevant documents first. Then uses the LLM to generate an answer. Prevents hallucinations by grounding answers in data.

✓ Uses LCEL for clean, composable chains

```
from langchain_openai import ChatOpenAI,
OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_community.document_loaders import
TextLoader
from langchain_text_splitters import
RecursiveCharacterTextSplitter

# Step 1: Load documents
loader = TextLoader("data.txt")
documents = loader.load()
```

PYTHON



Naved Khan
Senior Gen-AI Engineer

17/26

+ Follow

RAG Pipeline - Part 2

Continue: Split documents into chunks and create vector store.

PYTHON

```
# Split documents into chunks
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200 # Overlap helps maintain context
)
chunks = text_splitter.split_documents(documents)

# Step 2: Create vector store and retriever
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vector_store = FAISS.from_documents(chunks, embeddings)
retriever = vector_store.as_retriever(search_kwargs=
{"k": 3})
```



Naved Khan
Senior Gen-AI Engineer

18/26

+ Follow

RAG Pipeline - Part 3

Continue: Create RAG chain using LCEL.

PYTHON

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import
StrOutputParser
from langchain_openai import ChatOpenAI

# Step 3: Create RAG chain using LCEL
model = ChatOpenAI(model="gpt-4o-mini")
prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer the question using only the
following context:\n\n{context}\n\nIf you don't know the
answer, say so."),
    ("human", "{question}")
])
```



Naved Khan
Senior Gen-AI Engineer

19/26

+ Follow

RAG Pipeline - Part 4

Continue: Build and query the RAG chain.

PYTHON

```
def format_docs(docs):  
    """Format retrieved documents for the prompt."""  
    return "\n\n".join(doc.page_content for doc in docs)  
  
# Build RAG chain with LCEL  
rag_chain = (  
    {  
        "context": retriever | format_docs,  
        "question": RunnablePassthrough()  
    }  
    | prompt  
    | model  
    | StrOutputParser()  
)  
  
# Step 4: Query the RAG chain  
result = rag_chain.invoke("What does the document say  
about AI?")  
print(result)
```

20/26



Naved Khan

Senior Gen-AI Engineer

+ Follow

RAG Agent – Part 1

What is RAG Agent? Agent decides when to retrieve information. Can handle greetings, follow-ups without retrieval. Multiple LLM calls (one for decision, one for answer). Best for: Complex conversations, when retrieval may not always be needed.

PYTHON

```
from langchain.agents import create_agent
from langchain.tools import tool
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings

# Create retriever (assuming chunks already created)
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vector_store = FAISS.from_documents(chunks, embeddings)
retriever = vector_store.as_retriever()
```



Naved Khan
Senior Gen-AI Engineer

21/26

+ Follow

RAG Agent - Part 2

Continue: Create retriever tool and agent.

PYTHON

```
# Create retriever tool
@tool
def search_documents(query: str) → str:
    """Search documents for relevant information."""
    docs = retriever.invoke(query)
    return "\n\n".join([doc.page_content for doc in docs])

# Create RAG agent
agent = create_agent(
    model="gpt-4o-mini",
    tools=[search_documents],
    system_prompt="You are a helpful assistant. Use the search tool when you need information from documents."
)
```



Naved Khan
Senior Gen-AI Engineer

22/26

+ Follow

RAG Agent - Part 3

Continue: Invoke the agent and get results.

PYTHON

```
# Agent decides when to search
result = agent.invoke({
    "messages": [{"role": "user", "content": "What does
the document say about AI?"}]
})
print(result["messages"][-1].content)
```



Naved Khan
Senior Gen-AI Engineer

23/26

+ Follow

Multiple Tools - Part 1

What are Multiple Tools? Agents can use multiple tools simultaneously. Each tool performs a specific function. The agent decides which tool(s) to use based on the user's request. This enables complex workflows and multi-step operations.

PYTHON

```
from langchain.tools import tool
from langchain.agents import create_agent

@tool
def get_weather(city: str) → str:
    """Get the current weather for a city."""
    # In production, this would call a weather API
    return f"The weather in {city} is sunny, 72°F"

@tool
def get_time(city: str) → str:
    """Get the current time for a city."""
    # In production, this would get actual timezone time
    return f"The time in {city} is 3:00 PM"
```



Naved Khan
Senior Gen-AI Engineer

24/26

+ Follow

Multiple Tools - Part 2

Continue: Create agent with multiple tools and invoke it.

PYTHON

```
# Create agent with multiple tools
agent = create_agent(
    model="gpt-4o-mini",
    tools=[get_weather, get_time],
    system_prompt="You are a helpful assistant that can
check weather and time."
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "What's the
weather in New York?"}]
})
print(result["messages"][-1].content)
```



Naved Khan
Senior Gen-AI Engineer

25/26

+ Follow



THAT'S ALL!

Thank You for Reading!



Naved Khan

Senior Gen-AI Engineer

Save this cheatsheet for your LangChain projects.

Share with someone who needs it!



Save



Repost



Comment

+ Follow for More

26/26